

Programação de Sistemas Linux

Aula 05 – Chamadas de Sistema: Fundamentos

Hugo Marcondes

Departamento Acadêmico de Eletrônica
DAELN

hugo.marcondes@ifsc.edu.br



Objetivos da aula

Ao final desta aula, você será capaz de:

- ✓ Explicar o mecanismo de uma **chamada de sistema** (*syscall*)
- ✓ Diferenciar uma **syscall** de uma **função de biblioteca**
- ✓ Tratar erros de syscall com `errno`, `perror` e `strerror`
- ✓ Explicar por que a `libc` existe e o que ela adiciona sobre as syscalls cruas

Regra de hoje

Todo `printf`, todo `malloc`, todo `fopen` desce, em algum momento, para uma chamada de sistema. Hoje vamos ver esse caminho todo.

Notes

Notes



O mecanismo de syscall

Notes

Duas camadas, uma fronteira

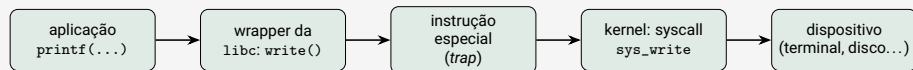
Lá na Aula 1 vimos a arquitetura do kernel: **espaço de usuário** vs. **espaço de kernel**.

- Seu programa roda em espaço de **usuário** — sem acesso direto a hardware, disco, rede
- Só o **kernel** pode tocar o hardware diretamente
- Uma **syscall** é a **única porta** controlada entre os dois mundos

Notes



O caminho de um printf



- A *trap* (instrução `syscall` no x86-64) muda a CPU para **modo kernel**
- O kernel executa `sys_write`, devolve o controle, a CPU volta ao **modo usuário**
- Tudo isso é **muito mais caro** que uma chamada de função comum – por isso `syscalls` custam tempo



Notes

Chamando uma syscall diretamente

```
1 #include <unistd.h>
2 #include <sys/syscall.h>
3
4 long n = syscall(SYS_write, STDOUT_FILENO, msg, tamanho);
5 long pid = syscall(SYS_getpid);
```

- `syscall()` é uma função **genérica** da libc: número da syscall + argumentos
- Ninguém escreve código assim no dia a dia – é só para **entender** o que `write()`, `getpid()` etc. fazem por baixo
- Na prática, sempre use os **wrappers** da libc (`write`, `getpid`...)



Notes

```
1 $ man 2 syscalls | head -30      # lista (parcial) das syscalls do Linux
2 $ ausyscall --dump | head        # numero -> nome, se o pacote existir
3 $ grep -c '[0-9]' /usr/include/asm-generic/unistd.h
```

- O kernel Linux tem hoje mais de **300** chamadas de sistema
- Cada uma tem um **número** fixo por arquitetura — é esse número que vai na instrução *trap*
- Vocês já usaram várias sem perceber: `open`, `read`, `write`, `fork`, `exit`...



Notes

Syscall vs. função de biblioteca

Notes

Definição

Uma **função de biblioteca** (libc) é código C comum, compilado junto com o seu programa (ou linkado dinamicamente). Uma **syscall** é um pedido ao kernel, via *trap*. `printf()` é biblioteca; lá no fundo, ela chama a syscall `write()`.

- `printf` formata a string, guarda num **buffer interno** e só eventualmente chama `write()`
- Várias chamadas de `printf` podem virar **uma única** syscall `write`



Notes

Medindo a diferença: 200 mil linhas

```
1 $ strace -c ./muitos_printf > /dev/null # 200000x printf("linha %d\n", i
   )
2 ...
3 100.00  0.006230   9      642    2 total  (608 chamadas de write)
4
5 $ strace -c ./muitos_write > /dev/null # 200000x write() manual, 1 por
   linha
6 ...
7 100.00  3.319794  16    20029    1 total  (200000 chamadas de write)
```

O buffer da libc está fazendo o trabalho pesado

`printf` virou só ~608 syscalls porque a libc **agrupa** a saída num buffer antes de gravar. Chamar `write()` **sem** buffer, uma vez por linha, dispara 200 000 syscalls – e o `strace` (que intercepta cada uma) deixa isso dolorosamente visível no tempo.



Notes

O que isso ensina

- Syscalls têm **custo fixo** (troca de modo, cópia de dados) — **menos chamadas, maiores**, geralmente, é mais rápido
- É exatamente por isso que a `stdio` (`printf`, `fwrite`...) usa buffer por padrão
- Quando **você** escreve I/O de baixo nível (como no Laboratório 02, com `read/write` direto), o **tamanho do seu próprio buffer** cumpre esse mesmo papel

Regra prática

Poucas chamadas de sistema, com buffers maiores, quase sempre vencem muitas chamadas pequenas — em I/O de arquivo, de rede, em qualquer lugar.



erro e tratamento de erros

Notes

Notes

- Por convenção, a maioria das syscalls devolve -1 em caso de erro
- O **motivo** do erro fica na variável global `errno` — definida em `<errno.h>`
- `errno` só é **válido logo após** a chamada que falhou — qualquer outra chamada pode sobrescrevê-lo

Erro clássico

Esquecer de checar o retorno de `open/read/write` e seguir usando um descritor de arquivo inválido (-1) como se fosse válido.



```
1 int fd = open("/caminho/que/nao/existe.txt", O_RDONLY);
2 if (fd == -1) {
3     printf("errno = %d (%s)\n", errno, strerror(errno));
4     perror("open");
5     return 1;
6 }
7
```

```
1 errno = 2 (No such file or directory)
2 open: No such file or directory
3
```

- `strerror(errno)` devolve a mensagem como `char *` — útil para logar
- `perror("open")` já imprime `"open: <mensagem>"` direto no `stderr`



Alguns erro que você vai ver sempre

Constante	Valor	Significado
ENOENT	2	arquivo/caminho não existe
EACCES	13	permissão negada
EEXIST	17	arquivo já existe (com O_CREAT O_EXCL)
EISDIR	21	era um diretório, esperava-se um arquivo
EMFILE	24	processo excedeu o limite de descritores abertos

man erro lista todos – não precisa decorar,
precisa saber consultar.



Notes

A libc como camada de conveniência

Notes

O problema

O número da syscall `write` não é o mesmo em todas as arquiteturas de CPU. E chamar `syscall()` cru, toda vez, pode ser caro!

A libc resolve isso:

- **Portabilidade:** mesma função `write()` funciona em x86-64, ARM, RISC-V... com números de syscall diferentes por baixo
- **Conveniência:** `fopen/fprintf` usam buffer, formatação, um `FILE *` em vez de um `int` cru
- **Padronização:** a mesma API (POSIX/ISO C) em praticamente todo Unix



	Syscall crua	Wrapper da libc
Abrir arquivo	<code>syscall(SYS_open, ...)</code>	<code>open()</code> / <code>fopen()</code>
Ler dados	<code>syscall(SYS_read, ...)</code>	<code>read()</code> / <code>fread()</code>
Buffer	nenhum	<code>fopen</code> : sim; <code>open</code> : não
Portabilidade	depende da arquitetura	garantida pela libc

Vocês vão alternar entre essas duas camadas o resto do semestre — saber qual usar é parte da disciplina.



Encerramento

Notes

Síntese da aula

Notes

Conceito	Pergunta que responde
Mecanismo de syscall	Como o processo pede algo ao kernel?
Syscall vs. libc	Essa função é código meu/da libc, ou vira kernel?
errno	Por que essa chamada falhou, e como eu descubro?
A libc	O que ganho usando fopen em vez de open cru?

Syscall, libc e errno: as três peças por trás de toda I/O que vocês já escreveram.



Atividade prática de hoje

No laboratório, vocês vão:

- 1 Chamar `write`, `getpid` e `getuid` via `syscall()` direta
- 2 Comparar o número de chamadas de sistema entre `printf` e `write` manual com `strace -c`
- 3 Provocar e tratar erros de `syscall` com `errno`, `perror` e `strerror`

Guia da atividade

Todas as instruções, código-fonte e critérios de avaliação estão no **guia de atividade prática** do Laboratório 04.



Notes

Próxima aula

Aula 6 – Inicialização do Sistema Linux (Boot)

- BIOS/UEFI e o bootloader (GRUB2)
- O que o kernel faz nos primeiros instantes
- `systemd`: *targets*, *units*, dependências
- Análise de logs de boot com `journald`

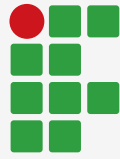
Bibliografia de apoio

- Manuais: `man 2 syscalls`, `man 3 errno`, `man 3 perror`, `man 3 strerror`
- The Linux Programming Interface (Michael Kerrisk) – capítulos 2–3
- HALLINAN, C. *Embedded Linux Primer* – capítulo sobre chamadas de sistema



Notes

That's all folks!



**INSTITUTO
FEDERAL**
Santa Catarina

Câmpus
Florianópolis



Notes

Notes
